

This material is partially based on the book draft “Reinforcement Learning: Foundations” by Shie Mannor, Yishay Mansour, and Aviv Tamar.

Reinforcement Learning (RL) is concerned with the design and analysis of algorithms that learn how to make decisions in arbitrary environments. A crucial aspect is that decisions must be taken sequentially and the algorithms must consider the implications of their decisions. Some practical problems to which RL has been applied are:

- Playing a board game, like Chess or Go.
- Controlling a robot to complete a certain task; for example, collecting items or rescuing people.
- Driving a car to a given destination.
- Keeping the parameters of a physical process in a safe and useful range of values (e.g., a controlled nuclear reaction that generates heat).
- Deciding which advertisement to show to each new visitor of a website.
- Managing a portfolio of stocks.

There are two features that typically distinguish RL applications from standard ML applications:

1. The decisions made by the algorithm may affect the outcome of future decisions.
2. When making a decision among a number of options, the algorithm typically observes only the outcome of the chosen option; the outcomes of the other options remain partially or totally unknown.

? Note that only one of these two features may appear in a RL application. For example, in advertising the first feature is missing as we may ignore the effect on the next visitor of showing an ad to the current visitor. On the other hand, in portfolio management the second feature is missing as we can simulate the outcome of an investment decision irrespective to the decision we actually made.

We use the word agent to refer to an algorithm operating in an environment. We can abstract the interaction between agent and environment through the mathematical notion of discrete-time (or sequential) decision process. At each time step, the agent is in a known state and the following happens:

1. The agent selects an action among those available in the current state and executes it
2. The agent's current state changes.

TRANSITION FROM STATE S TO
TRANSITION FUNCTION

WORLD AGENT

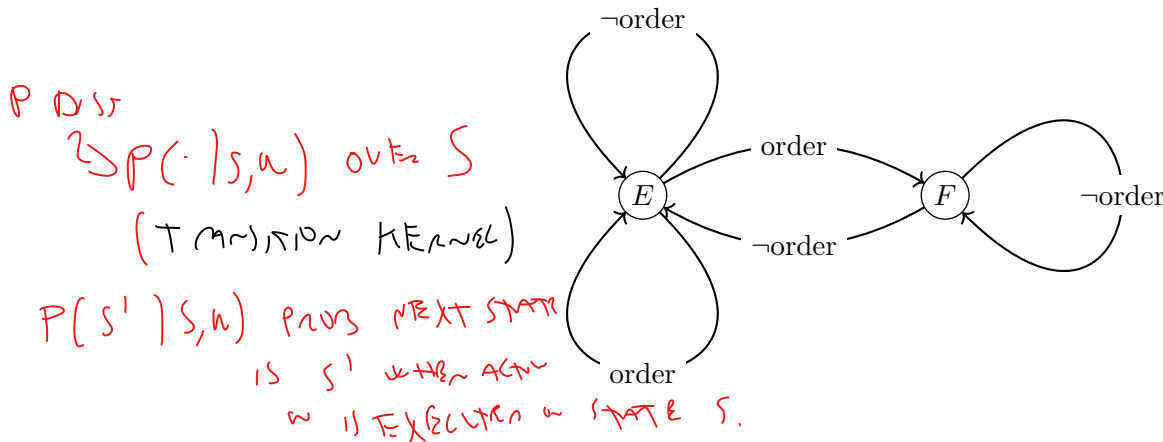
1
DISC. TIME DECISION PROBLEMS
INTERACTING AGENT/ENV.

$A(s)$ ACTION AVAILABLE AT STATE s

In the first part of this course we focus on finite decision processes, where the set $\mathcal{S}$ of states and the set $\mathcal{A}$ of actions are both finite. In the second part, we will consider the case in which the state set can be arbitrarily large. For each $s \in \mathcal{S}$, we use $\mathcal{A}(s)$ to denote the actions available in state s .

Example 1 (Finding the shortest path in a weighted directed graph.) This is one of the most fundamental algorithmic problems on graphs and can be viewed as a deterministic decision process where the states are the nodes of the graph and the actions available in each state correspond to the outgoing edges. The edge weight is the cost of traversing the edge. Given a start state and a goal state, the agent must find the sequence of actions corresponding to the path of minimum total cost from the start node to the goal node. When the graph is fully known, the optimal sequence of actions can be found using, for example, Dijkstra's algorithm for single source and single destination shortest path.

Example 2 (Managing an inventory.) A retailer sells one item at the time from a certain set of goods. The retailer has either no items in stock (state E) or exactly one item in stock (state F). If the state is E , the retailer can order one item from the supplier (action "order"). If the item is immediately sold, then the next state is again E ; otherwise, the next state is F . If the state is F and the item in stock gets sold, then the next state is E . If the item remains in stock, then the retailer pays for holding the stock. Note that action " $\neg$ order" (do not order a new item) is the only action possible in state F . If action " $\neg$ order" is executed in state E , then the retailer may miss a sale if someone willing to buy shows up. Note also that the action "order" executed in state E



can lead to state E (if the ordered item is immediately sold) or to state F (if the ordered item is not sold immediately). The actual next state (E or F) thus depends on the current demand for the item, which is typically not known until the item is put on sale.

In order to capture the uncertainty in the effect of an action on the environment, we allow the state transition to be stochastic, where the distribution over the future state depends on both the current state and the action selected by the agent. In symbols, for every $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$ there exists a probability distribution $p(\cdot | s, a)$ over $\mathcal{S}$ (called **transition kernel**) where $p(s' | s, a)$ is the probability that the next state is s' when action a is executed in state s .

TRAJECTORY: STATE, ACTION, P ~~AND~~ NEXT STATE s' WITH $\sim$ ACTION EXECUTED ON STATE s

The triple $\langle \mathcal{S}, \mathcal{A}, \{p(\cdot | s, a) : s \in \mathcal{S}, a \in \mathcal{A}(s)\} \rangle$ defines a **Markov Decision Process (MDP)**. The **Markovian property** refers to the fact that the next state only depends on the current state and the selected action, and not on the previous states and actions. *no any knowledge requires*

The behavior of the agent interacting with an MDP is specified by a **control policy**. A deterministic control policy is a map from states to actions. The stochastic sequence $(s_0, a_0, s_1, a_1, \dots)$ of states and actions generated by a policy interacting with the MDP from some initial state $s_0 \in \mathcal{S}$ is called a **trajectory**. In this trajectory, $s_{t+1} \sim p(\cdot | s_t, a_t)$ for all $t \geq 0$ where the notation $\sim$ means that s_{t+1} is a random variable drawn from $p(\cdot | s_t, a_t)$. We also write $\mathbb{P}(s_{t+1} = s | s_t, a_t) = p(s | s_t, a_t)$ where $\mathbb{P}(\cdot | \cdot)$ denotes conditional probability.

A deterministic policy $\pi = (\pi_t)_{t \geq 0}$ is a sequence of mappings π_t , where each π_t maps any possible history (i.e., past trajectory including the current state) $\mathbf{h}_t = (s_0, a_0, \dots, s_{t-1}, a_{t-1}, s_t)$ to an action $a_t \in \mathcal{A}(s_t)$. A deterministic **Markov policy** $\pi = (\pi_t)_{t \geq 0}$ can be written, for all $t \geq 1$, as $\pi_t : \mathcal{S} \rightarrow \mathcal{A}$ such that $a_t = \pi_t(s_t)$. In other words, the action selected at time t only depends on the current state s_t and not on the history. Control policies can be **randomized**. Then $\pi_t(\mathbf{h}_t)$ is a probability distribution over $\mathcal{A}(s_t)$ and $a_t \sim \pi_t(\cdot | \mathbf{h}_t)$.

Recall that a **Markov chain** on a state space $\mathcal{S}$ with initial state s_0 is a random walk $s_0, s_1, \dots$ over $\mathcal{S}$ such that

$$\mathbb{P}(s_t = s' | s_0, \dots, s_{t-1}) = \mathbb{P}(s_t = s' | s_{t-1})$$

16-11-17 H. H. H. 070

for all $s' \in \mathcal{S}$ and for all $t \geq 1$.

If we fix a (randomized) Markov policy and an initial state $s_0 \in \mathcal{S}$, then the stochastic sequence $(s_t)_{t \geq 0}$ of states traversed by the policy is a **Markov chain with transition probabilities**

$$\begin{aligned} \text{ALT } \mathbb{P}(s_{t+1} = s' | s_t = s) &= \sum_{a \in \mathcal{A}(s)} \mathbb{P}(s_{t+1} = s', a_t = a | s_t = s) \quad \text{So, all poss. ACT, 0-1, 1-6, take 1 state} \\ &= \sum_{a \in \mathcal{A}(s)} \mathbb{P}(s_{t+1} = s' | s_t = s, a_t = a) \mathbb{P}(a_t = a | s_t = s) \\ &= \sum_{a \in \mathcal{A}(s)} p(s' | s, a) \pi_t(a | s) \end{aligned}$$

From now on, to simplify notation we assume $\mathcal{A}(s) \equiv \mathcal{A}$ for all $s \in \mathcal{S}$. All the results we show remain true even when this assumption is lifted.

Recall that an MDP models an environment, while the policy defines a behavior of the agent. In general, we would like the agent to behave in a way that is best possible for the given environment. We can achieve that by assigning values to the policies and instructing the agent to learn the policy with the highest value.

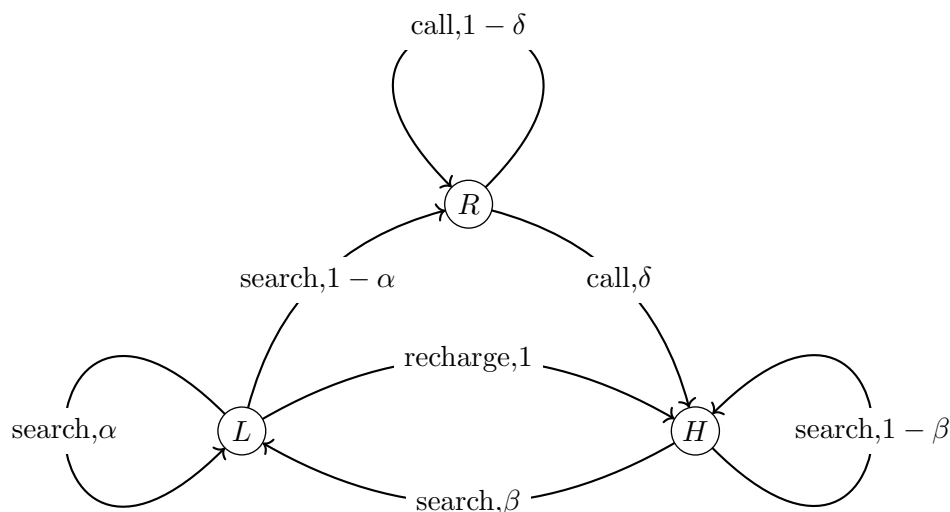
As each policy corresponds to a stochastic trajectory $(s_0, a_0, s_1, a_1, \dots)$ of states and actions, we can assign a value to each trajectory and then assign to the policy the expectation of this value. In the MDP framework, this is done by assigning a reward $r_t(s, a)$ to each state-action pair, where $r_t : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is a time-dependent **reward function**, and then by summing up the rewards along the trajectory. In many practical examples, the interaction between the agent and the environment ends after a given number of rounds. In this case, we also use a **terminal reward function** $r_{\text{end}} : \mathcal{S} \rightarrow \mathbb{R}$ assigning a value to the state reached at the end of the interaction (in a game, the

$r_t(s, a)$ REWARD

$r_{\text{end}} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$

terminal reward reflects whether the agent won the game whereas the rewards assigned to state-action pairs traversed during the game may help the agent distinguish “good” moves from “bad” moves).

Example 3 (Recycling robot.) A robot roams an office to collect empty cans that have to be recycled. The robot can be in three states depending on the battery charge: H (high charge), L (low charge), R (rescue me). Our goal is to have the robot search for cans as often as possible while entering the rescue state as few times as possible. Hence, we assign a positive reward to the search action (from any state) and a negative reward to the call action (from the R state). Since recharging is a neutral action, we give it a zero reward. In the figure, the label a, p on an edge (s, s')



indicates the action name a and the transition probability $p = p(s' | s, a)$.

The role of the reward is similar to that of the loss in supervised machine learning: it is the main signal through which the agent learns a desired behavior. As we said earlier, we instruct agents to maximize their cumulative reward along the trajectory followed on the MDP. The value of the cumulative reward is called **return**. Given a policy with stochastic trajectory $(s_0, a_0, s_1, a_1, \dots)$ on a given MDP, we can define the return according to the following two **evaluation criteria**:

- Finite deterministic horizon: $\mathbb{E} \left[\sum_{t=0}^{T-1} r_t(s_t, a_t) + r_{\text{end}}(s_T) \right]$ (interaction ends after T steps)
- Infinite horizon: $\lim_{T \rightarrow \infty} \mathbb{E} \left[\frac{1}{T} \sum_{t=0}^{T-1} r_t(s_t, a_t) \right]$ (interaction never ends)

DIFFERENT Goal States

We also consider the important case of a **stochastic horizon**, i.e., when the horizon T can be a random variable. To define the return using a stochastic horizon, we identify a subset $\mathcal{G} \subseteq \mathcal{S}$ of **goal states** in the MDP and stop the interaction as soon as the agent reaches any state in $\mathcal{G}$. Denoting by $s_0, s_1, \dots$ the stochastic trajectory of states realized by the agent's policy, we define $T = \min \{t \geq 0 : s_t \in \mathcal{G}\}$. The expected return of a stochastic Markov policy π according to the stochastic horizon criterion is the same as the finite horizon case, with the only difference that T is now a random variable.

It is easy to see that the finite deterministic horizon criterion is a special case of the stochastic horizon criterion.

In many applications, for instance when the agent controls a physical process, there are no specific goal states and the interaction goes on forever. However, the interaction may stop at any point of time because of certain events like a fault or an external cause. The presence of random stopping points in an MDP without goal states can be implemented with a small modification to the MDP so that the horizon becomes stochastic rather than infinite. Given an MDP $\langle \mathcal{S}, \mathcal{A}, p(\cdot | s, a) \rangle$ without goal states, we add a single goal state s_G and define a new transition kernel p' defined by $p'(s_G | s, a) = 1 - \gamma$ and $p'(s' | s, a) = \gamma p(s' | s, a)$ for each $s, s' \in \mathcal{S} \setminus \{s_G\}$, $a \in \mathcal{A}$, and for some $0 < \gamma < 1$. The resulting evaluation criterion, which is a special case of stochastic horizon, is known as the **discounted infinite horizon** (or γ -discounted horizon). In practice, the discounted infinite horizon assumes that the interaction may stop at any point of time with probability $1 - \gamma$.

For all $t \geq 0$, assuming $s_{t-1} \neq s_G$, the probability of ~~not~~ stopping at time t is

$$\mathbb{P}(s_t \neq s_G | s_{t-1}, a_{t-1}) = \gamma \sum_{s' \neq s_G} p(s' | s_{t-1}, a_{t-1}) = \gamma$$

for any action $a_{t-1} \in \mathcal{A}$. Therefore, given any policy executing actions $a_0, a_1, \dots$, the probability of stopping at time $t + 1$ or later assuming $s_0 \neq s_G$ is

$$\mathbb{P}(T > t) = \mathbb{P}(s_1 \neq s_G, \dots, s_t \neq s_G | s_0, a_0) = \prod_{\tau=1}^t \mathbb{P}(s_\tau \neq s_G | s_{\tau-1}, a_{\tau-1}) = \gamma^t$$

Since the event $T = t$ does not depend on the specific trajectory of states and actions, we can fix any infinite trajectory $\mathbf{h} = (s_0, a_0, s_1, a_1, \dots)$ and write the return on this trajectory in expectation with respect to the randomness of T as

$$\begin{aligned} \sum_{t=0}^{\infty} \sum_{\tau=0}^t r_\tau(s_\tau, a_\tau) \mathbb{P}(T = t+1) &= \sum_{\tau=0}^{\infty} r_\tau(s_\tau, a_\tau) \sum_{t=\tau}^{\infty} \mathbb{P}(T = t+1) \\ &= \sum_{\tau=0}^{\infty} r_\tau(s_\tau, a_\tau) \mathbb{P}(T > \tau) \\ &= \sum_{t=0}^{\infty} \gamma^t r_t(s_t, a_t) \end{aligned}$$

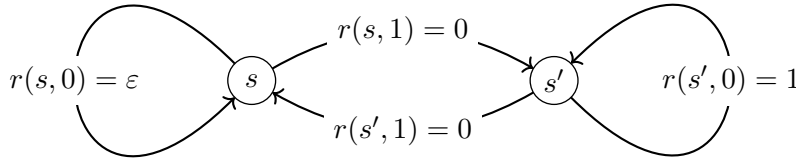
Taking expectation with respect to the randomness in the trajectory, we obtain that the expected return with respect to the discounted infinite horizon is

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t(s_t, a_t) \right]$$

Suboptimality of the greedy policy. The greedy policy is a deterministic policy defined as

$$\pi_t(s) = \operatorname{argmax}_{a \in \mathcal{A}} r_t(s, a)$$

It is easy to construct examples of MDP where this policy is never optimal. In the deterministic MDP below, if $s_0 = s$, the greedy policy deterministically achieves a reward of ε in each step. On the other hand, the policy π , such that $\pi(s) = 1$ and $\pi(s') = 0$, achieves zero reward in the first step and reward 1 for all the remaining steps.



We now show that in order to maximize the stochastic horizon criterion, it is sufficient to consider Markov policies. To avoid infinite sums of rewards, we consider MDPs where any policy eventually reaches a goal state with probability one. Let μ be a probability distribution over the initial state s_0 , $\mathbb{P}(s_0 = s) = \mu(s)$, and let $q_t^\pi(s, a) = \mathbb{P}^\pi(s_t = s, a_t = a)$ be the **occupancy measure** evaluated at (s, a) . This is the distribution of (s_t, a_t) under strategy π (with initial state distribution μ). To compute $q_t^\pi(s, a)$ we sum the probabilities of all trajectories $(s_0, a_0, s_1, a_1, \dots)$ that end in a goal state and such that $s_t = s$ and $a_t = a$. Let $r_t(s, \cdot) = r_{\text{end}}(s)$ for any goal state $s \in \mathcal{G}$. Then the expected total reward is:

$$\mathbb{E} \left[\sum_{t=0}^{T-1} r_t(s_t, a_t) + r_{\text{end}}(s_T) \right] = \mathbb{E} \left[\sum_{t=0}^T r_t(s_t, a_t) \right] = \sum_{t=0}^{\infty} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} r_t(s, a) q_t^\pi(s, a)$$

This shows that any two policies that induce the same occupancy measure for all $t \geq 0$ must have the same expected total reward.

Theorem 4 (Sufficiency of Markov policies) *Given an MDP with initial state distribution μ , consider a (possibly stochastic) non-Markov policy π . Then there exists a stochastic Markov policy π' such that $q_t^\pi = q_t^{\pi'}$ for all $t \geq 0$.*

PROOF. For every $t \geq 0$, every state $s \in \mathcal{S}$, and every $a \in \mathcal{A}$ let

$$\pi'_t(a | s) = \frac{q_t^\pi(a, s)}{\sum_{a' \in \mathcal{A}} q_t^\pi(a', s)}$$

Clearly, π' is Markov because $\pi'_t(\cdot | s)$ only depends on s and not on the history. We prove that $q_t^\pi = q_t^{\pi'}$ by induction on $t \geq 0$. Let $\mathbb{P}^\pi$ be the probability of states and actions when π is run on the MDP. For $t = 0$,

$$q_0^{\pi'}(a, s) = \mathbb{P}^{\pi'}(a_0 = a | s_0 = s) \mu(s) = \frac{q_0^\pi(a, s)}{\sum_{a' \in \mathcal{A}} q_0^\pi(a', s)} \mu(s) = q_0^\pi(a, s)$$

because $\sum_{a' \in \mathcal{A}} q_0^\pi(a', s) = \mu(s)$ by definition. Now assume $q_{t-1}^\pi = q_{t-1}^{\pi'}$ holds. Note that

$$\begin{aligned} \mathbb{P}^{\pi'}(s_t = s) &= \sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} \mathbb{P}^{\pi'}(s_{t-1} = s', a_{t-1} = a') p(s | s', a') \\ &= \sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} q_{t-1}^{\pi'}(a', s') p(s | s', a') \\ &= \sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} q_{t-1}^\pi(a', s') p(s | s', a') && \text{(by inductive hyp.)} \\ &= \mathbb{P}^\pi(s_t = s) \end{aligned}$$

where p is the transition kernel of the MDP (which does not depend on the policy). Therefore,

$$\begin{aligned} q_t^{\pi'}(a, s) &= \mathbb{P}^{\pi'}(a_t = a | s_t = s) \mathbb{P}^{\pi'}(s_t = s) \\ &= \pi'_t(a | s) \mathbb{P}^\pi(s_t = s) \\ &= \frac{q_t^\pi(a, s)}{\sum_{a' \in \mathcal{A}} q_t^\pi(a', s)} \mathbb{P}^\pi(s_t = s) \\ &= q_t^\pi(a, s) \end{aligned}$$

and this concludes the proof. $\square$

So, from now on, without loss of generality we only consider Markov policies.

We now state another interesting fact: randomization cannot increase the expected return of a Markov policy. Let

$$V_\gamma^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t(s_t, a_t) \right]$$

be the expected return on the discounted infinite horizon of a Markov policy π with initial state $s_0 = s$.

Theorem 5 (Sufficiency of deterministic Markov policies) *Given an MDP, consider a stochastic Markov policy π . Then there exists a deterministic Markov policy π' such that $V_\gamma^\pi(s) = V_\gamma^{\pi'}(s)$ for all $s \in \mathcal{S}$.*